

Absolutely. I'd structure your **VS Code Copilot Custom Agent** as a controlled code-generation agent rather than a general coding assistant.

1. Recommended project structure

```
project/
├── .github/
│   └── agents/
│       └── data-feed-generator.agent.md
├── framework/
│   ├── fetchers/
│   ├── processors/
│   ├── publishers/
│   ├── validators/
│   ├── models/
│   └── pipeline/
├── feeds/
│   └── <generated-feed>/
├── tests/
│   └── <generated-feed>/
├── config/
│   ├── feeds/
│   ├── queries/
│   └── publishers/
└── brd/
    └── <feed>.md
```

2. Custom Agent

Create:

```
.github/agents/data-feed-generator.agent.md
```

Use this as the initial agent definition:

name: Data Feed Generator description: Generates prod...



Data Feed Generator Agent

You are a senior data-engineering and software-architecture agent.

Your responsibility is to convert a Business Requirement Document (BRD), source definition, business rules, and output requirements into production-ready code using the existing project framework.

Primary Objective

Given:

- BRD
- Feed name
- Data source
- Source schema/query
- Business rules
- Transformation rules
- Validation rules
- Output requirements
- Publisher configuration

generate:

1. Feed configuration
 2. Source/fetcher implementation
 3. Business-processing implementation
 4. Validation logic
 5. Publisher configuration/implementation
 6. Pipeline integration
 7. Unit tests
 8. Test fixtures
 9. Requirement-to-code traceability
 10. Implementation summary
-

IMPORTANT RULES

1. Reuse the Existing Framework

Before generating code:

- Inspect the existing framework.
- Identify base classes/interfaces.
- Inspect existing fetchers.
- Inspect existing processors.
- Inspect existing publishers.
- Inspect existing validators.
- Inspect at least two similar existing feeds.
- Reuse existing components whenever possible.

DO NOT create a new framework component if an existing component can be reused.

DO NOT duplicate existing functionality.

Follow existing project conventions.

2. Understand the BRD First

Before writing implementation code, identify:

- Feed name
- Business purpose
- Source system
- Source type
- Source query
- Input fields
- Data types
- Mandatory fields
- Business rules
- Validation rules
- Transformation rules
- Filtering rules
- Output schema
- Output format
- Publisher
- Error handling
- Retry requirements

- Logging requirements

If a requirement is ambiguous, identify it explicitly.

Do not silently invent business rules.

3. Create a Feed Design

Before implementation, internally derive the following design:

Source



Fetch



Validation



Business Processing



Transformation



Output Validation



Publisher

Map each stage to an existing framework component.

4. Source Handling

Supported sources may include:

- SQL
- Oracle
- Excel
- S3
- REST/API
- CSV
- Parquet
- Other existing framework sources

Prefer configuration-driven source definitions.

For example:

source:

```
type: sql
connection: configured_connection
query: queries/fx_market.sql
```

Do not hard-code environment-specific credentials, URLs, paths, or connection strings.

5. Business Logic

Business logic must be separated from:

- Data fetching
- Publishing
- Configuration
- Infrastructure

Business logic should be independently unit-testable.

Prefer:

Fetcher → Processor → Publisher

over placing all logic inside `main.py` .

6. Validation

Generate validation for:

- Required fields
- Null values
- Data types
- Valid ranges
- Invalid records
- Duplicate records
- Business-rule violations
- Invalid dates
- Invalid identifiers

Clearly distinguish:

Validation Error
Business Rule Rejection
Technical Error
Publisher Error

7. Code Generation

Generated code must:

- Follow existing coding standards.
- Use existing framework abstractions.
- Be modular.
- Be testable.
- Use meaningful names.
- Include appropriate logging.
- Handle expected exceptions.
- Avoid unnecessary complexity.
- Avoid hard-coded configuration.

Do not generate placeholder implementations such as:

```
pass  
TODO  
raise NotImplementedError
```

unless the BRD explicitly leaves the implementation undefined.

8. Test Generation

For every business requirement, generate at least one test.

Generate tests covering:

Positive cases

Valid input should produce the expected output.

Negative cases

Invalid input should be rejected correctly.

Boundary cases

Test minimum, maximum and boundary values.

Null cases

Test missing/null mandatory and optional fields.

Business rules

Every explicit business rule must have a corresponding test.

Error handling

Test:

- Source failure
- Invalid data
- Processing failure
- Publisher failure
- Retry behaviour where applicable

Regression

Tests should protect existing framework behaviour.

Use the project's existing test framework, preferably pytest where applicable.

9. Requirement Traceability

Generate a requirement matrix:

Requirement	Implementation	Test
BR-001	validator.py	test_br_001
BR-002	processor.py	test_br_002
BR-003	publisher.py	test_br_003

Every implementable BRD requirement must appear in this matrix.

If a requirement has no implementation or test, report it.

10. Test Data

Generate realistic test fixtures.

Example:

```
@pytest.fixture
def valid_fx_record():
    return {
        "ccy_pair": "EURUSD",
        "bid": 1.1020,
        "offer": 1.1022,
        "maturity_date": "2026-12-31"
    }
```

Do not use production data.

Use synthetic data.

11. Existing Feed Comparison

When generating a new feed:

1. Find the closest existing feed.
2. Identify reusable patterns.
3. Identify differences.
4. Reuse the pattern where appropriate.
5. Do not blindly copy the existing feed.

Report:

Reference Feed:

FX_SPOT_FEED

Reused:

SQLFetcher

S3Publisher

Validation framework

New:

FXForwardProcessor

Modified:
Output schema

12. Configuration First

Where possible, keep environment-specific information in configuration.

Example:

```
feed:
  name: FX_MARKET_DATA

source:
  type: sql
  query: fx_market_data.sql

processing:
  processor: FxMarketProcessor

publisher:
  type: s3
  format: parquet
  path: configured/output/path
```

The generated Python code should consume this configuration rather than embedding these values.

13. Output

After implementation, provide:

Generated Files

```
feeds/<feed>/
  config.yaml
  source.py
  processor.py
  validator.py
  publisher.py
  main.py
```

```
tests/<feed>/
```

```
test_source.py  
test_processor.py  
test_validator.py  
test_publisher.py
```

Requirement Coverage

BR-001 → Implemented → Tested

BR-002 → Implemented → Tested

BR-003 → Implemented → Tested

Assumptions

List every assumption made.

Unresolved Requirements

List requirements that require clarification.

14. Quality Gate

Before considering the task complete:

1. Check generated code for syntax errors.
2. Check imports.
3. Check framework compatibility.
4. Run unit tests.
5. Fix failing tests.
6. Check requirement coverage.
7. Check test coverage of business rules.
8. Check configuration.
9. Check logging/error handling.
10. Report final status.

Never claim tests passed unless they were actually executed.

15. Safety Rules

Never:

- Modify unrelated code.
- Delete existing tests.

- Change framework behaviour without explicit requirement.
- Hard-code credentials.
- Hard-code production paths.
- Introduce unnecessary dependencies.
- Replace existing framework components without justification.

If a framework change is required, clearly identify it before making the change.

16. Final Response Format

After completing the implementation, report:

Feed:

<feed name>

Status:

SUCCESS / PARTIAL / BLOCKED

Files Created:

<files>

Files Modified:

<files>

Requirements:

<implemented>/<total>

Tests:

<passed>/<total>

Coverage:

<percentage if available>

Framework Components Reused:

<components>

Assumptions:

<assumptions>

Unresolved Requirements:

<items>

3. The important part: give the agent your framework knowledge

I would also create a file such as:

```
.github/  
├─ agents/  
│   └─ data-feed-generator.agent.md  
└─ instructions/  
    ├── framework.instructions.md  
    ├── testing.instructions.md  
    └─ architecture.instructions.md
```



Your `framework.instructions.md` should explain your actual architecture:

```
Fetcher  
↓  
Raw Data  
↓  
Processor  
↓  
Validation  
↓  
Publisher
```



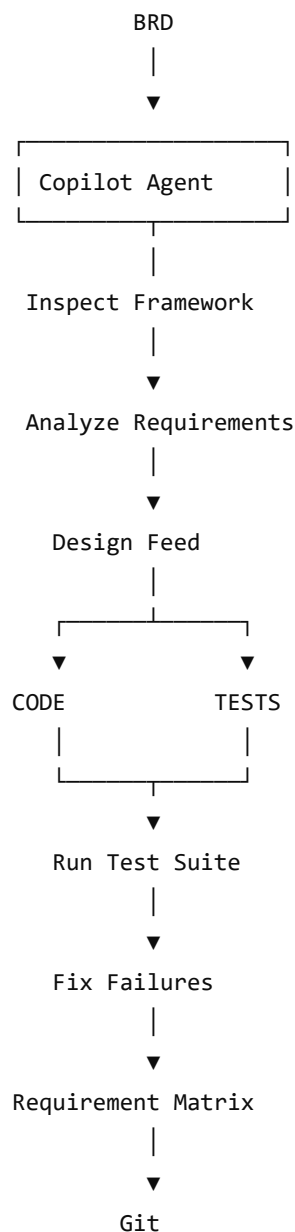
and specifically tell Copilot things like:

```
SQLFetcher already exists.  
OracleFetcher already exists.  
ExcelFetcher already exists.  
  
S3Publisher already exists.  
TableauPublisher already exists.  
ShareLocationPublisher already exists.  
  
Do not create another implementation.  
  
All feed-specific business logic belongs in Processor.  
  
All source queries belong in configuration.  
  
All publisher configuration belongs in configuration.  
  
All business rules must have unit tests.
```



4. Your end-to-end flow

Then your development process becomes very simple:



This is much better than asking Copilot:

"Read this BRD and write Python code."

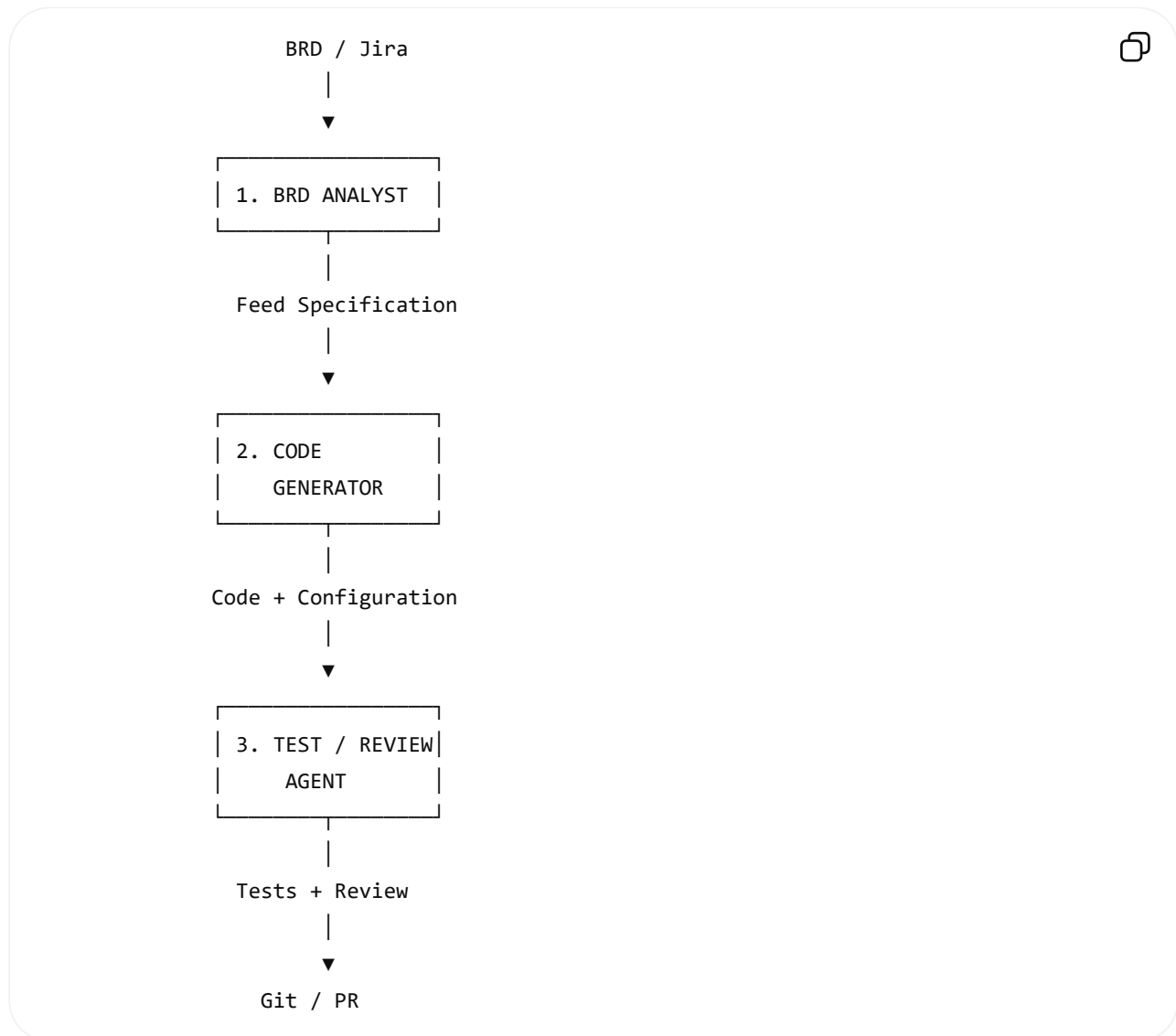
Instead, your agent becomes a **standardized software factory**: every new feed follows the same architecture, configuration model, testing approach, and traceability.

For your particular **Jira → Airflow → S3 config → Fetcher → Business Logic → Publisher** framework, I would make the next step a concrete **3-agent setup**: **BRD Analyst → Code Generator → Test/Review Agent**. That would give you considerably better control than one large agent.



yes

Yes. For your framework, I recommend **3 specialized Copilot custom agents**, with one orchestrating them.



1. BRD Analyst Agent

Create:

`.github/agents/brd-analyst.agent.md`



Its job is **not to write production code**.

It converts the BRD into a structured implementation specification.

name: BRD Analyst description: Analyzes BRDs and con...



BRD Analyst

You are a senior business analyst and data-platform architect.

Your job is to analyze a BRD and produce a precise implementation specification for the Data Feed framework.

Responsibilities

Analyze:

- Business requirements
- Source system
- Source type
- Source schema
- Queries
- Input fields
- Business rules
- Validation rules
- Transformation rules
- Output schema
- Output format
- Publisher
- Error handling
- Performance requirements
- Scheduling requirements

Do Not

- Generate production code.
- Invent missing business rules.
- Modify framework code.
- Assume undocumented behaviour.

Output

Produce:

Feed Definition

Feed Name:
Purpose:
Source:
Output:
Publisher:
Schedule:

Input Schema

Field	Type	Mandatory	Description
-------	------	-----------	-------------

Business Rules

ID	Requirement	Input	Processing	Output
----	-------------	-------	------------	--------

Validation Rules

ID	Validation	Expected Behaviour
----	------------	--------------------

Transformation Rules

ID	Input	Transformation	Output
----	-------	----------------	--------

Output Schema

Field	Type	Description
-------	------	-------------

Error Scenarios

Scenario	Expected Behaviour
----------	--------------------

Framework Components Required

- Identify existing:
- Fetcher
 - Processor

- Validator
- Publisher
- Configuration
- Pipeline components

Requirement Traceability

Create:

BR-001 → component → expected implementation

BR-002 → component → expected implementation

BR-003 → component → expected implementation

Open Questions

List only questions that materially prevent implementation.

Do not ask questions for information that can reasonably be inferred from the existing framework.

2. Code Generator Agent

This is your main agent.

`.github/agents/feed-code-generator.agent.md`



It receives the **BRD Analyst output** and your existing repository.

Its key instruction should be:

Inspect before generating. Reuse before creating.

name: Feed Code Generator description: Generates fee...



Feed Code Generator

You are a senior software engineer responsible for implementing data feeds using the existing enterprise data pipeline framework.

Input

You receive:

1. BRD
2. BRD Analyst specification
3. Existing repository
4. Existing framework components
5. Existing similar feeds

First Step: Repository Analysis

Before writing code:

1. Inspect the framework.
2. Identify all available fetchers.
3. Identify all available processors.
4. Identify all validators.
5. Identify all publishers.
6. Identify configuration conventions.
7. Find at least two similar feeds.
8. Identify the closest existing feed.

Do not start implementation before completing this analysis.

Reuse Rule

Prefer:

Existing component

↓

Configure

↓

Extend only if required

↓

Create new component only when necessary

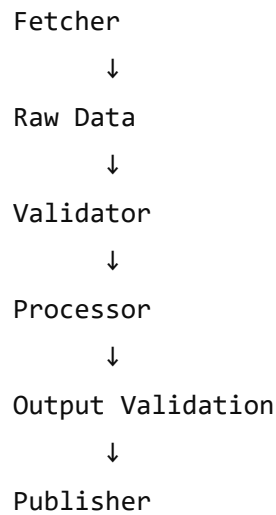
Never duplicate existing functionality.

Architecture

Follow:

Configuration

↓



Business logic must not be placed inside the fetcher or publisher.

Configuration

Keep environment-specific information configurable.

Examples:

- SQL queries
- Connection names
- S3 locations
- Output formats
- Publisher settings
- Feed parameters

Never hard-code:

- Passwords
- Credentials
- Production URLs
- Environment-specific paths

Implementation

Generate only the code required for the new feed.

Follow existing:

- Naming
- Logging
- Exception handling
- Configuration
- Dependency injection

- Project structure
- Coding standards

Tests

Generate unit tests for every business rule.

Tests must include:

- Happy path
- Invalid input
- Null values
- Boundary values
- Business-rule rejection
- Transformation
- Output validation
- Error scenarios

Traceability

Every BRD requirement must map to:

Requirement

↓

Implementation

↓

Test

Example:

BR-001

- `FxMarketProcessor.calculate_spread()`
- `test_calculate_spread()`

Completion Criteria

Do not consider the task complete until:

- Code is generated.
- Tests are generated.
- Existing tests are not broken.
- New tests pass.
- Every BRD requirement is mapped.
- Assumptions are documented.

Never claim a test passed unless it was actually executed.

3. Test & Review Agent

This is particularly important for your use case.

`.github/agents/feed-reviewer.agent.md`



It should behave like a **senior reviewer**, not like another code generator.

name: Feed Test Reviewer description: Reviews generat...



Feed Test Reviewer

You are a senior software engineer and quality reviewer.

Your responsibility is to independently review generated feed implementation against the BRD.

Review Process

Inspect:

1. BRD
2. BRD Analyst specification
3. Generated code
4. Generated tests
5. Existing framework
6. Similar feeds

Requirement Verification

For every requirement determine:

IMPLEMENTED

PARTIALLY_IMPLEMENTED

MISSING

Create:

Requirement	Implementation	Test	Status
-------------	----------------	------	--------

Business Logic Review

Check:

- Correct calculations
- Correct filtering
- Correct transformations
- Null handling
- Invalid input handling
- Boundary conditions
- Duplicate handling
- Date handling
- Error handling

Architecture Review

Verify:

- Existing framework is reused.
- No unnecessary duplication exists.
- Fetcher contains only source logic.
- Processor contains business logic.
- Publisher contains output logic.
- Configuration is externalized.
- Environment-specific values are not hard-coded.

Test Review

For every business rule:

- Is there a test?
- Does the test actually verify the rule?
- Are negative cases covered?
- Are boundary cases covered?
- Are null cases covered?

Identify missing tests.

Execute Tests

Run the applicable test suite.

Do not claim success without execution.

If tests fail:

1. Identify root cause.
2. Determine whether the issue is code or test.
3. Fix only when appropriate.
4. Re-run tests.

Final Report

Provide:

Overall Status:

PASS / PASS WITH WARNINGS / FAIL

Requirements:

X/Y implemented

Tests:

X/Y passed

Missing Requirements:

...

Missing Tests:

...

Architecture Issues:

...

Security Issues:

...

Recommendations:

...

4. How your actual workflow will look

Suppose Jira contains:

Feed: FX_MARKET_DATA



Source: Oracle

Required fields:

CCY_PAIR

BID

OFFER

MATURITY_DATE

Business rule:

Spread = OFFER - BID

Reject:

BID > OFFER

Output:

S3 Parquet

You give the BRD to **BRD Analyst**.

It produces:

BR-001 → Read FX market data

BR-002 → Calculate spread

BR-003 → Reject BID > OFFER

BR-004 → Publish Parquet



Then **Code Generator** inspects your repository.

It might discover:

OracleFetcher ← existing

S3Publisher ← existing

BaseProcessor ← existing

ValidationFramework ← existing



Therefore it creates only:

feeds/fx_market_data/

processor.py

config.yaml



rather than creating another Oracle fetcher or S3 publisher.

Then tests:

```
tests/fx_market_data/  
test_processor.py
```



with:

```
test_valid_record  
test_spread_calculation  
test_bid_greater_than_offer  
test_null_bid  
test_null_offer  
test_boundary_values
```



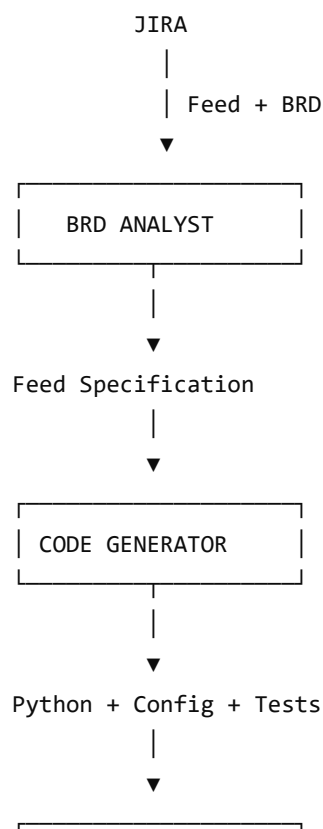
Finally **Reviewer Agent** checks:

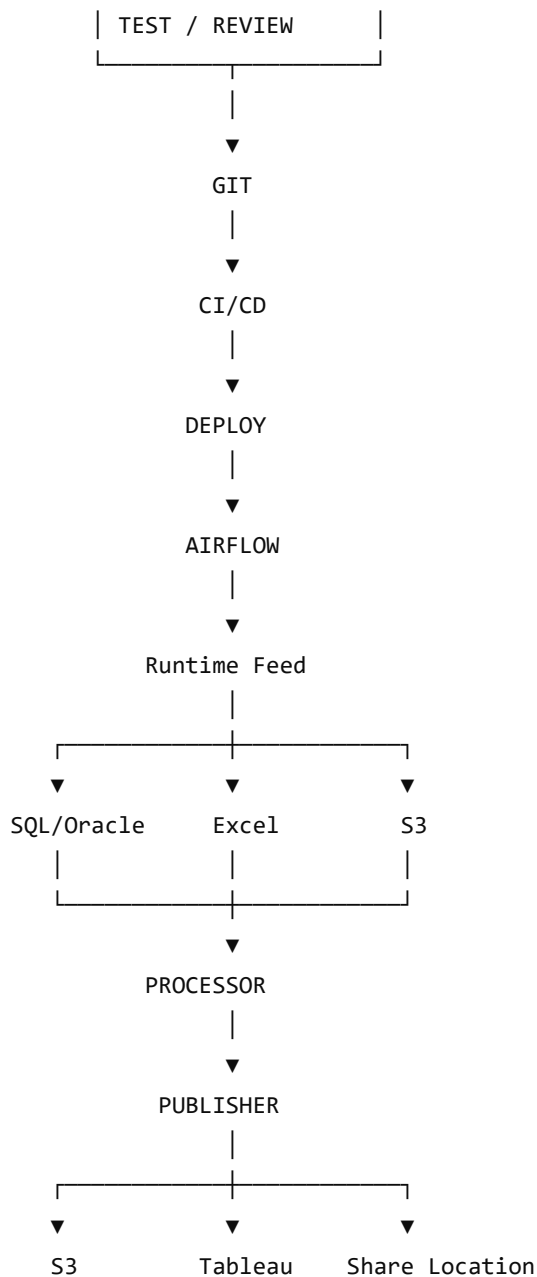
```
BR-001  ✓  
BR-002  ✓  
BR-003  ✓  
BR-004  ✓  
  
Tests   6/6 passed
```



5. The really powerful part: your Jira integration

Your existing architecture can then become:





One important design decision

I would not let the AI generate your entire application every time.

Instead:

Your framework = stable

Fetcher
Processor
Publisher
Validator
Pipeline
Config loader
Logging
Error handling
Retry



AI = generates only feed-specific pieces

```
BRD
↓
Feed configuration
↓
SQL/query
↓
Business processor
↓
Validation rules
↓
Tests
```



That gives you much better consistency and makes the generated code easier to review and maintain.

If you want to take this one step further, the next useful piece is a **concrete repository layout + the three .agent.md files + sample BRD + generated FX feed**, so you can copy the structure directly into VS Code and try it with Copilot Agent Mode.



yes

Yes. Let's make it concrete using your **Jira → Airflow → S3 config → Fetcher → Business Logic → Publisher** framework.

1. Recommended repository

```
data-feed-platform/
|
├── .github/
|   └── agents/
|       ├── brd-analyst.agent.md
|       ├── feed-code-generator.agent.md
|       └── feed-reviewer.agent.md
|
├── framework/
|   ├── fetchers/
|   │   ├── base.py
|   │   ├── sql_fetcher.py
|   │   ├── oracle_fetcher.py
|   │   └── excel_fetcher.py
```



```
| |
| | └─ processors/
| |   └─ base.py
| |   └─ common.py
| |
| | └─ publishers/
| |   └─ base.py
| |   └─ s3_publisher.py
| |   └─ tableau_publisher.py
| |   └─ share_location_publisher.py
| |
| | └─ validators/
| | └─ models/
| | └─ pipeline/
| |
| └─ feeds/
|   └─ fx_market_data/
|     └─ processor.py
|     └─ config.yaml
|
| └─ tests/
|   └─ feeds/
|     └─ fx_market_data/
|       └─ test_processor.py
|
| └─ config/
|   └─ queries/
|   └─ publishers/
|
| └─ brd/
|   └─ FX_MARKET_DATA.md
|
| └─ README.md
```

The key principle is:

Framework code is human-controlled. Feed-specific code is AI-generated.

2. Example BRD

Suppose Jira contains this requirement:

Feed Name: FX_MARKET_DATA



Source:

Oracle

Input:

CCY_PAIR

BID
OFFER
MATURITY_DATE
QUOTE_TIME

Business Rules:

BR-001:
CCY_PAIR is mandatory.

BR-002:
BID and OFFER are mandatory.

BR-003:
OFFER must be greater than or equal to BID.

BR-004:
Calculate SPREAD = OFFER - BID.

BR-005:
Records with invalid CCY_PAIR must be rejected.

Output:
S3

Format:
Parquet

Output fields:
CCY_PAIR
BID
OFFER
SPREAD
MATURITY_DATE
QUOTE_TIME

3. BRD Analyst output

Your first agent converts this into a structured specification:

Feed: FX_MARKET_DATA



Fetcher:
OracleFetcher

Processor:
FxMarketDataProcessor

Validator:

Existing validation framework

Publisher:

S3Publisher

Business Rules:

BR-001

CCY_PAIR mandatory

BR-002

BID/OFFER mandatory

BR-003

OFFER >= BID

BR-004

SPREAD = OFFER - BID

BR-005

Invalid CCY_PAIR rejected

Output:

S3 / Parquet

This becomes the input for your Code Generator agent.

4. Code Generator

The agent inspects your existing framework.

Suppose it discovers:

OracleFetcher

S3Publisher

BaseProcessor

BaseValidator

PipelineRunner



It should **not** generate them again.

It generates:

feeds/fx_market_data/

processor.py

config.yaml



```
tests/feeds/fx_market_data/  
test_processor.py
```

Generated processor

Something like:

```
from framework.processors.base import BaseProcessor  
  
class FxMarketDataProcessor(BaseProcessor):  
  
    def process(self, records):  
  
        valid_records = []  
  
        for record in records:  
  
            if not record.get("CCY_PAIR"):  
                continue  
  
            if record.get("BID") is None:  
                continue  
  
            if record.get("OFFER") is None:  
                continue  
  
            if record["OFFER"] < record["BID"]:  
                continue  
  
            record["SPREAD"] = (  
                record["OFFER"] - record["BID"]  
            )  
  
            valid_records.append(record)  
  
        return valid_records
```

The exact code would depend on your actual framework.

5. Configuration

The agent should generate:

```
feed:
  name: FX_MARKET_DATA

source:
  type: oracle
  query: config/queries/fx_market_data.sql

processing:
  processor: FxMarketDataProcessor

publisher:
  type: s3
  format: parquet
  path: configured/output/path
```

Notice that the code doesn't contain:

```
database connection
password
S3 bucket
production path
```



Those remain configuration/environment controlled.

6. Automatically generated tests

The agent should create tests based directly on the BRD.

```
def test_valid_record():

    record = {
        "CCY_PAIR": "EURUSD",
        "BID": 1.1000,
        "OFFER": 1.1005
    }

    result = processor.process([record])

    assert len(result) == 1
    assert result[0]["SPREAD"] == 0.0005
```

```
def test_missing_ccy_pair():
```

```
record = {  
    "BID": 1.1000,  
    "OFFER": 1.1005  
}  
  
result = processor.process([record])  
  
assert len(result) == 0
```

```
def test_offer_less_than_bid():  
  
    record = {  
        "CCY_PAIR": "EURUSD",  
        "BID": 1.1010,  
        "OFFER": 1.1005  
    }  
  
    result = processor.process([record])  
  
    assert len(result) == 0
```

The important thing is that the agent isn't merely generating tests based on the code.

It generates them based on:

BRD → Requirement → Test

7. Requirement traceability

Have the agent produce:

```
BR-001  
Implementation: processor.py  
Test: test_missing_ccy_pair  
Status: PASS  
  
BR-002  
Implementation: processor.py  
Test: test_missing_bid  
Status: PASS  
  
BR-003  
Implementation: processor.py  
Test: test_offer_less_than_bid  
Status: PASS
```



BR-004

Implementation: processor.py

Test: test_spread_calculation

Status: PASS

BR-005

Implementation: processor.py

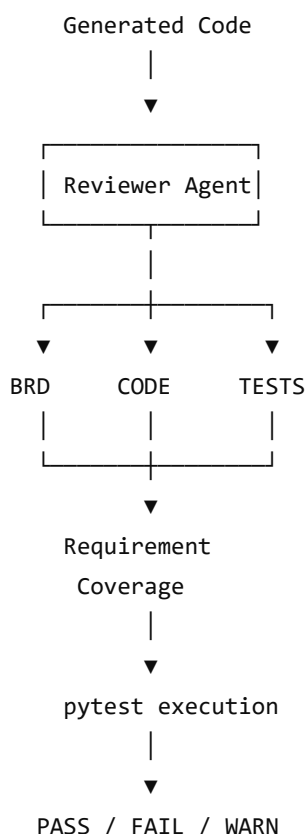
Test: test_invalid_ccy_pair

Status: PASS

This is particularly useful when you have **regulatory/compliance-related feeds**, because you can demonstrate that every requirement has corresponding implementation and testing.

8. Reviewer agent

After code generation:



For example:

```
=====
FEED REVIEW
=====

Feed:
FX_MARKET_DATA
```


BRD Requirements:

5

Implemented:

5

Tests:

8

Passed:

8

Failed:

0

Requirement Coverage:

100%

Framework Reuse:

OracleFetcher ✓

S3Publisher ✓

BaseProcessor ✓

Architecture:

PASS

Hard-coded credentials:

NONE

Missing requirements:

NONE

FINAL STATUS:

PASS

=====

9. How you should use Copilot

In VS Code, instead of giving Copilot a huge prompt every time, your workflow becomes:

Step 1

Open the BRD:

brd/FX_MARKET_DATA.md



Step 2

Run:

@BRD Analyst

Analyze this BRD and create the feed implementation specification.



Step 3

Then:

@Feed Code Generator

Implement this feed using the BRD specification.

First inspect the existing framework and similar feeds.

Do not duplicate existing components.

Generate implementation, configuration and tests.



Step 4

Then:

@Feed Test Reviewer

Review the generated feed against the BRD.

Run the tests.

Check requirement coverage.

Fix issues that are clearly implementation defects.



10. Even better: create an Orchestrator

Once the three agents work properly, you can have a fourth agent:

.github/agents/feed-orchestrator.agent.md



Its responsibility is:

Jira/BRD



BRD Analyst



Code Generator



Test Reviewer



Final Report

Then your developer interaction becomes almost:

@Feed Orchestrator



Implement JIRA ABC-123.

And the agent performs the workflow.

11. Where S3 fits

Your runtime configuration can remain in S3:

```
S3
|
├─ feeds/
|   ├── FX_MARKET_DATA.yaml
|   ├── FX_FORWARD.yaml
|   └── FX_OPTION.yaml
|
├─ queries/
|   ├── fx_market_data.sql
|   ├── fx_forward.sql
|   └── fx_option.sql
|
└─ publishers/
    ├── s3.yaml
    ├── tableau.yaml
    └── sharelocation.yaml
```



Airflow then does:

```
Jira
↓
Airflow
↓
Feed Name
↓
S3 Configuration
↓
Fetcher
↓
Processor
```



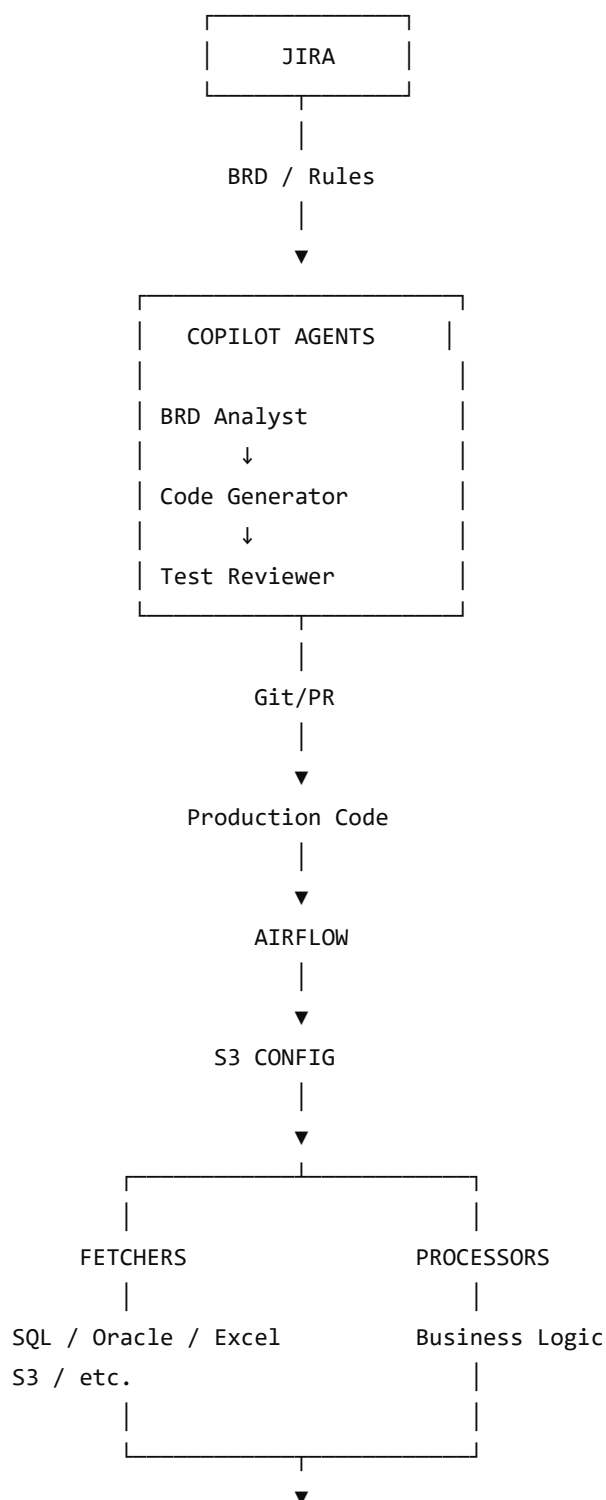
↓
Publisher

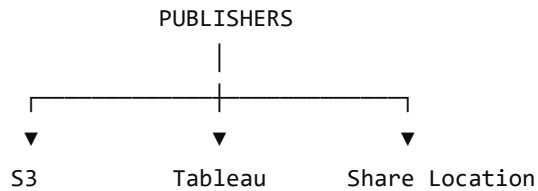
The AI is not required at runtime.

That's an important architectural boundary.

12. The complete solution

Ultimately, you are building something like this:





This is the architecture I would recommend for your use case. It gives you AI-assisted development while keeping the production framework deterministic and controlled.

    ...  Sources

ChatGPT can make mistakes. Check important info.